

AI FAQ Assistant API

Project Description

AI FAQ Assistant API is an AI-powered question-answering system designed to provide quick and relevant answers to frequently asked questions. The system allows users to enter their questions through a web interface, sends the request to the backend API, processes the question using an AI model, and returns an appropriate answer.

The application provides a simple and user-friendly interface where users can submit questions related to a particular service, organization, product, or general FAQ information. The backend API manages user requests, validates the input, communicates with the AI service, and returns the generated response in a structured format.

The system reduces the need for users to manually search through large amounts of FAQ information and provides an interactive way to obtain answers. Secure API key management, input validation, error handling, and structured API responses are implemented to improve reliability and security.

Scenarios

- **Scenario 1: General FAQ Question**

A user visits the AI FAQ Assistant website and enters a frequently asked question such as “**What services are available?**” The system sends the question to the FAQ Assistant API, processes it using the AI model, and displays a relevant answer to the user.

- **Scenario 2: Multiple User Questions**

A user asks different questions one after another, such as “**How can I register?**”, “**What documents are required?**”, and “**How can I contact support?**” The API processes each request and returns the corresponding answers without requiring the user to search manually.

- **Scenario 3: Invalid or Empty Question**

A user submits the form without entering a question. The backend validates the request and returns an appropriate error message asking the user to enter a valid question instead of sending an empty request to the AI service.

Technical Architecture

Description

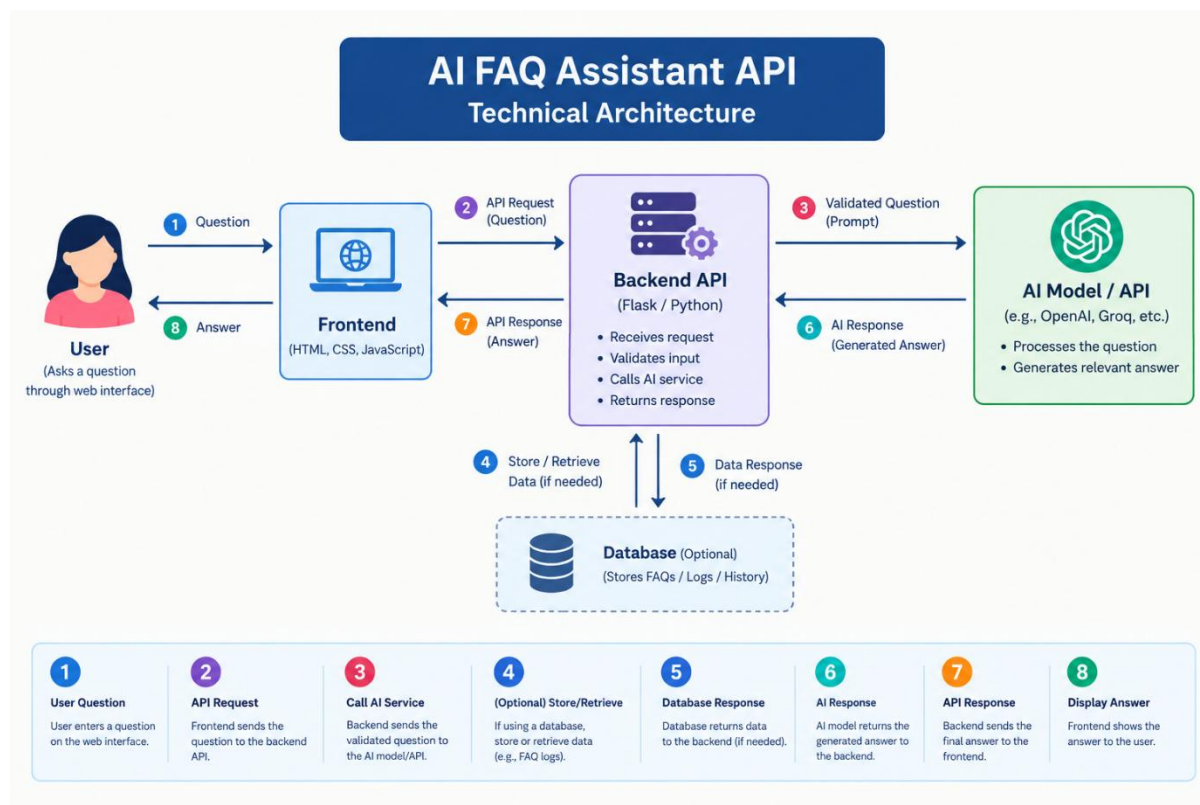
The AI FAQ Assistant API follows a modular architecture consisting of a **frontend**, **backend API**, and **AI service**.

The frontend provides a responsive web interface where users can enter their questions. The backend API receives the question, validates the input, constructs the required request, and communicates with the AI service. The AI service processes the question and generates a suitable answer. The backend then sends the response back to the frontend for display.

The architecture can be represented as:

User → Frontend → Backend API → AI Model/API → Backend API → Frontend → Answer

If a database is used in your project, an **ER Diagram** should also be included, as the sample documentation notes.



Pre-requisites

- **Python Programming Knowledge:** Required for backend development.
- **Flask Framework Knowledge:** Required if Flask is used for the API.
- **REST API Knowledge:** Understanding of GET/POST requests and JSON responses.
- **HTML, CSS, and JavaScript Skills:** Required for developing the web interface.
- **AI API Familiarity:** Required to integrate the selected AI model/API.
- **Postman:** Used for API testing.
- **Git and GitHub:** Used for version control.
- **Development Environment:** VS Code or another suitable code editor.
- **Environment Variables:** Used to securely store API keys.

The sample similarly lists framework, API, programming, Git, and development-environment prerequisites.

Project Workflow

Milestone 1: Model Selection and Architecture

Activity 1.1: Select the AI Model/API

Research suitable AI models or APIs that can understand user questions and generate relevant FAQ responses.

Consider:

- Response accuracy
- Response speed
- API availability
- Cost and usage limits
- Ease of integration
- Response format

Activity 1.2: Define the Application Architecture

Define how the frontend, backend API, and AI service communicate with each other.

The basic workflow is:

User Question → Frontend → Backend API → AI Service → Generated Answer → Frontend

Activity 1.3: Design API Endpoints

Define the API endpoints required for the project.

Example:

POST /ask

This endpoint receives a user's question and returns the AI-generated answer.

Activity 1.4: Set Up Development Environment

Install the required programming language, framework, libraries, and dependencies.

Create a project structure containing:

- app.py
- requirements.txt
- .env
- templates/
- static/

Milestone 2: Core Functionalities Development

Activity 2.1: Develop FAQ Question Module

Create a question input form that allows users to enter their FAQ questions.

The system should:

- Accept the user's question.
- Validate the input.
- Send the question to the backend API.
- Display the generated answer.

Activity 2.2: Implement Backend API

Create the backend API to receive and process user questions.

The API should:

1. Receive the question.
2. Validate the question.
3. Send the question to the AI service.
4. Receive the AI response.
5. Return the answer in JSON format.

Example response:

```
{  
  "success": true,  
  "question": "What services are available?",  
  "answer": "Our services include..."  
}
```

Milestone 3: API Development

This milestone focuses on creating the main API logic and ensuring that the AI FAQ Assistant API processes requests correctly.

Activity 3.1: Create the /ask Endpoint

Create a POST endpoint that accepts the user's question.

Example request:

```
{  
  "question": "How can I register?"
```

```
}
```

The backend receives the request and processes the question.

Activity 3.2: Input Validation

Validate the incoming request before processing it.

The API should return an error when:

- The question is empty.
- The question is missing.
- The input is longer than the permitted limit.
- Invalid data is received.

Activity 3.3: AI API Integration

The backend sends the validated question to the selected AI service.

The AI service processes the question and generates a relevant response.

Activity 3.4: Return Structured Response

The backend returns the generated answer to the frontend in JSON format.

Example:

```
{  
  "success": true,  
  "answer": "You can register by completing the registration form."  
}
```

Milestone 4: Frontend Development

Activity 4.1: Design the User Interface

Develop a simple and user-friendly FAQ Assistant webpage using HTML and CSS.

The page should contain:

- Project title
- Question input box
- Ask button
- Loading indicator
- Answer section
- Error message section

Activity 4.2: JavaScript API Integration

Use JavaScript and the Fetch API to send the user's question to the backend.

The frontend should:

1. Capture the question.
2. Send a POST request to /ask.
3. Wait for the API response.
4. Extract the answer.
5. Display the answer dynamically.

Activity 4.3: Error Handling

Display a meaningful message when:

- The question is empty.
- The API is unavailable.
- The AI service returns an error.
- An unexpected server error occurs.

Milestone 5: Testing and Deployment

Activity 5.1: API Testing

Use **Postman** to test the AI FAQ Assistant API.

Test:

- Valid questions
- Empty questions
- Long questions
- Multiple questions
- Invalid requests
- API response time
- Error responses

Activity 5.2: Local Testing

Run the backend application locally and verify that the website and API work correctly.

Example:

`http://127.0.0.1:5000`

Test the complete workflow:

Question → API → AI Service → Answer

Activity 5.3: Performance Testing

Test the API under different request loads.

Check:

- Average response time
- Maximum response time
- Throughput
- Error rate
- CPU utilization
- Memory utilization

The sample documentation also treats local testing and deployment as separate activities within the deployment milestone.

Milestone 6: Deployment

The AI FAQ Assistant API can first be tested locally and then deployed using the hosting/deployment method selected for the project.

Before deployment:

- Install all required dependencies.
- Configure environment variables.
- Keep API keys secure.
- Test all API endpoints.
- Verify frontend and backend communication.
- Test error handling.

If using a tunneling service such as Ngrok, the local backend can be exposed through a public URL, similar to the deployment approach demonstrated in the sample.

Exploring the Website's Web Pages

Home / FAQ Assistant Page

Description

The home page serves as the main interface of the AI FAQ Assistant. It provides a simple interface where users can enter their questions and receive AI-generated answers.

The page contains:

- AI FAQ Assistant title
- Question input field
- Ask button
- Loading indicator
- Answer display section

Input Section

Description

The input section allows users to enter their questions into a text box.

The user enters a question and clicks the **Ask Question** button. The frontend sends the question to the backend API for processing.

Example:

Question:

“What services are available?”

The API processes the question and returns the answer.

Results Section

Description

The results section displays the answer returned by the AI FAQ Assistant API.

After the API successfully processes the question, the generated answer is dynamically displayed on the webpage.

The section should also display an appropriate error message when the API cannot process the request.

API Endpoint

POST /ask

Purpose:

Processes a user's FAQ question and returns an AI-generated answer.

Request

```
{  
  "question": "How can I register?"  
}
```

Response

```
{
```

```
"success": true,  
"question": "How can I register?",  
"answer": "You can register by completing the registration process."  
}
```

Conclusion

The **AI FAQ Assistant API** is an AI-powered question-answering system designed to provide users with quick and relevant answers to frequently asked questions. The project combines a user-friendly web interface, backend API, input validation, AI integration, structured JSON responses, and API testing.

The system reduces the effort required to search through FAQ information by providing an interactive question-answering experience. The project also demonstrates important concepts such as **API development, AI integration, frontend-backend communication, JSON processing, error handling, testing, and deployment.**

Future improvements can include **FAQ database integration, user authentication, conversation history, multilingual support, voice-based questions, admin management of FAQ content, and analytics.**



Contabo GmbH

VPS 4 – 8 GB, from \$5.28/mo

4.6 on Trustpilot. The Best Value Cloud VPS on Earth. Free Firewall & DDoS Protection.

Ad

More options

can you send me as a word doc

It seems like I can't do more advanced data analysis right now. Please try again later.

You've reached your data analysis limit. Upgrade to ChatGPT Go or try again after 11:05 AM.

Upgrade to Go